AI-Powered Smart Email Assistant

A full-stack application built around real generative AI

Team NEURAL NEXUS

Category Artificial Intelligence

Technology domain Generative AI and Natural Language Processing

Delivery Local Windows application with a Python backend

Report date November 1, 2026

Implementation status

All five AI workflows and seven pages are implemented. Offline checks and browser checks passed. Live model execution requires the evaluator's Gemini API key and has not been verified without it.

This report describes the delivered code. Screenshots show the actual application and sample inputs, not generated output evidence.

Report guide

Part	Coverage
Foundations	1 Title page, 2 Abstract, 3 Introduction, 4 Problem statement
Design	5 Existing system, 6 Proposed system, 7 Objectives, 8 Functional requirements, 9 Non-functional requirements
Implementation	10 Architecture, 11 Technology stack, 12 Modules, 13 Workflow, 14 UI screenshots, 15 Implementation details
Evaluation	16 Advantages, 17 Limitations, 18 Future enhancements, 19 Conclusion, 20 References

2. Abstract

NEURAL NEXUS is a local web application that assists users with drafting and refining email communication. It uses a real Google Gemini model to generate emails and subject lines, correct grammar, improve writing, adjust tone and compose context-aware replies. The system accepts natural-language input and returns structured results containing one or two drafts, a short intent summary, explanations and communication suggestions.

A custom HTML, CSS and JavaScript frontend communicates with a Python FastAPI backend. A centralized service uses Google's official GenAI SDK and a dedicated prompt for each operation. Pydantic validates both input and output. The application stores drafts only in browser memory and provides copy, TXT export and regeneration controls. It has no email account access or send capability.

The deliverable includes source code, pinned dependencies, a Windows setup guide, automated checks, a live demonstration script and a presentation. API access and semantic output quality remain external checks until a valid user key is configured.

3. Introduction

Email remains a common medium for academic and professional communication. Users often know what they want to say but struggle with structure, tone, grammar or concise expression. A generative language model can interpret context and produce candidate wording. This project applies that capability in a focused workspace where the user retains responsibility for the final message.

4. Problem statement

A student or professional may need to ask for an extension, request information, follow up or reply to a scheduling message. Writing each email manually requires attention to audience, tone and factual details. Basic spellchecking can identify some errors but does not necessarily understand the desired response or restructure a message in context.

The problem addressed is how to turn a short instruction or imperfect message into a coherent email while preserving the user's intent. The system must make AI central, remain simple to configure and avoid creating an illusion that it can read an inbox or send email.

5. Existing system

The comparison below describes common approaches rather than a benchmark against named commercial products. Many modern commercial tools also use AI. This project's purpose is to demonstrate an integrated, explainable implementation suitable for a college evaluation.

Approach	Strength	Gap addressed here
Manual composition	Full user control	Repeated effort to structure and refine each message
Static email templates	Quick reusable structure	Limited adaptation to new facts and intended responses
Basic spelling tools	Surface-level corrections	Limited task-specific generation and reply planning
General chat assistant	Flexible language generation	Requires repeated prompting and manual output handling

Design constraints

• All intelligent outputs must come from Gemini. No hardcoded replacement emails are permitted.

• No Gmail authentication, inbox access or sending functionality belongs in this release.

• Local operation should require only a Python environment, internet access and one secret configuration value.

6. Proposed system

The proposed system organizes five related AI operations behind a consistent interface. Users select a tool, enter content, choose available preferences and submit one request. The backend validates the request and sends a task-specific prompt to Gemini. A structured schema keeps subjects, bodies and explanations separate. Only validated results reach the UI.

The results panel displays a readable email and the model's brief explanation. Users can inspect alternatives, copy a draft or download it. Regeneration makes a new request only on an explicit click. A local history list retains up to eight results in the current tab, helping users revisit work without a database.

7. Project objectives

• Generate complete emails and relevant subjects from natural-language instructions.

• Correct grammar and improve writing while preserving factual meaning.

• Transform wording into Formal, Professional, Friendly, Polite and Casual tones.

• Generate replies using both incoming context and the user's intended response.

• Make AI involvement visible through processing states, metadata and explanations.

• Handle invalid input, missing configuration and provider failures honestly.

• Deliver a consistent responsive interface, documentation and a repeatable demonstration.

Success criteria

Every major operation must reach the shared AI service. A failed request must show a useful error without fabricated results. Form controls and productivity actions must work consistently. Real model quality must be assessed on intent, factual consistency, subject relevance, grammar, tone and clarity, rather than an invented accuracy percentage.

8. Functional requirements

ID	Requirement	Implementation
FR-01	Generate email and subject	Generate endpoint, recipient/purpose/tone inputs
FR-02	Correct language errors	Grammar prompt with minimal-edit constraints
FR-03	Improve message quality	Clarity and professional rewriting prompt
FR-04	Transform tone	Five supported tone choices
FR-05	Generate contextual reply	Incoming email plus required reply intent
FR-06	Review alternatives	One or two drafts in a single request
FR-07	Use results	Copy, TXT download, regenerate and clear
FR-08	Guide and inform	Samples, counts, empty/loading/error states
FR-09	Navigate and revisit	Seven pages and tab-memory recent results

9. Non-functional requirements

Quality	Implementation
Security	Environment secrets, no credential output, CSP, escaped text, local host validation and origin/session checks
Privacy	No database, localStorage or email content logs. Google still processes submitted text
Reliability	Schema validation, bounded input/output, timeouts, safe provider errors and two concurrent requests
Usability	Responsive layout, labels, keyboard focus, skip link, readable text and consistent controls
Maintainability	Separate configuration, schemas, prompts, services and frontend components
Cost control	One SDK attempt per action, no automatic retries, no model calls for simple UI operations

10. System architecture

Layer	Responsibilities and boundaries
Browser UI	Form handling, module navigation, temporary state, safe plain-text rendering, copy and download
FastAPI boundary	POST /api/{operation}, request token and origin validation, Pydantic input validation, body and concurrency limits
AI service	Dedicated feature methods, task prompt selection, official Google GenAI client, timeout and error mapping
Gemini model	Context and intent interpretation, subject/body generation, rewriting and concise explanations
Response contract	Pydantic output validation and metadata, then JSON returned to the browser

Data movement

The browser sends only the submitted form fields to the local Python server. The server adds task instructions and sends them to Google. Gemini returns structured content, which the server validates before returning it. The browser keeps the result in memory until a refresh or explicit clearing action. TXT downloads are an intentional user export.

Trust boundaries

The API key never crosses into the browser. Incoming email text is treated as untrusted data inside the prompt. The frontend escapes model text rather than executing HTML. These controls reduce common risks but cannot guarantee resistance to every prompt-injection attempt or prevent model mistakes.

Why there is no database

Persistence adds setup and privacy responsibilities that do not help the agreed initial scope. Memory-only drafts provide enough continuity for a demonstration. The backend does not write user email content to files during normal application operation.

11. Technology stack

Component	Choice
Backend	Python 3.12, FastAPI 0.162.0, Uvicorn 0.54.0
AI integration	Google GenAI Python SDK 2.34.0
Default model	gemini-3.5-flash-lite, configurable in .env
Validation/configuration	Pydantic 2.14.2, python-dotenv 1.2.4
Frontend	HTML5, custom CSS, browser JavaScript modules
Testing	pytest 9.2.0, httpx test client, jsdom 30.1.3
Artifacts	ReportLab PDF and editable PowerPoint deliverables

12. Module descriptions

Email generator: interprets natural-language instructions, audience, purpose and selected tone. It produces a complete candidate email and suitable subject.

Grammar correction: requests minimal spelling, punctuation and grammar changes. It preserves the original tone rather than applying a separate rewrite style.

Writing improvement: improves clarity, concision and professional structure while retaining information and commitments.

Tone adjustment: changes wording and phrasing to the selected tone while preserving facts, dates and intent.

Reply generator: separates incoming email content from user intent, preventing the incoming sender from choosing the user's response. It drafts a suitable reply with a subject.

Each module also returns a brief intent description, improvements and suggestions. These are generated summaries, not internal reasoning or a verified edit-by-edit audit.

13. Application workflow

Step	User action or system behavior
1	Start the local server and open the dashboard. The UI checks configuration without contacting Gemini.
2	Choose an AI tool. Type input or load a sample into the form.
3	Select available preferences. Choose one or two draft alternatives.
4	Click the action button. The UI disables duplicate submissions and shows processing status.
5	The backend checks limits and constructs the task prompt. The SDK sends one request.
6	The backend validates the response. Invalid or unavailable results produce an honest error.
7	Review subject, body, intent and suggestions. Check all facts and commitments.
8	Copy, download, switch alternatives or explicitly regenerate. Review recent results from the dashboard.

Error workflow

Missing credentials produce a setup message before any external request. Quota errors identify rate limits or billing/quota checks. Access and model errors point to server configuration. Timeouts suggest shorter input. Empty or malformed output never becomes a pretend successful email. A failed regeneration leaves an earlier successful result available and displays the new error separately.

State lifecycle

Form values and outputs remain available while navigating within the tab. Refreshing starts a new session and clears content. Clearing the form removes that module's current content. Clearing recent history removes history entries. Neither operation can delete TXT files already downloaded by the user.

14. UI screenshots

Dashboard

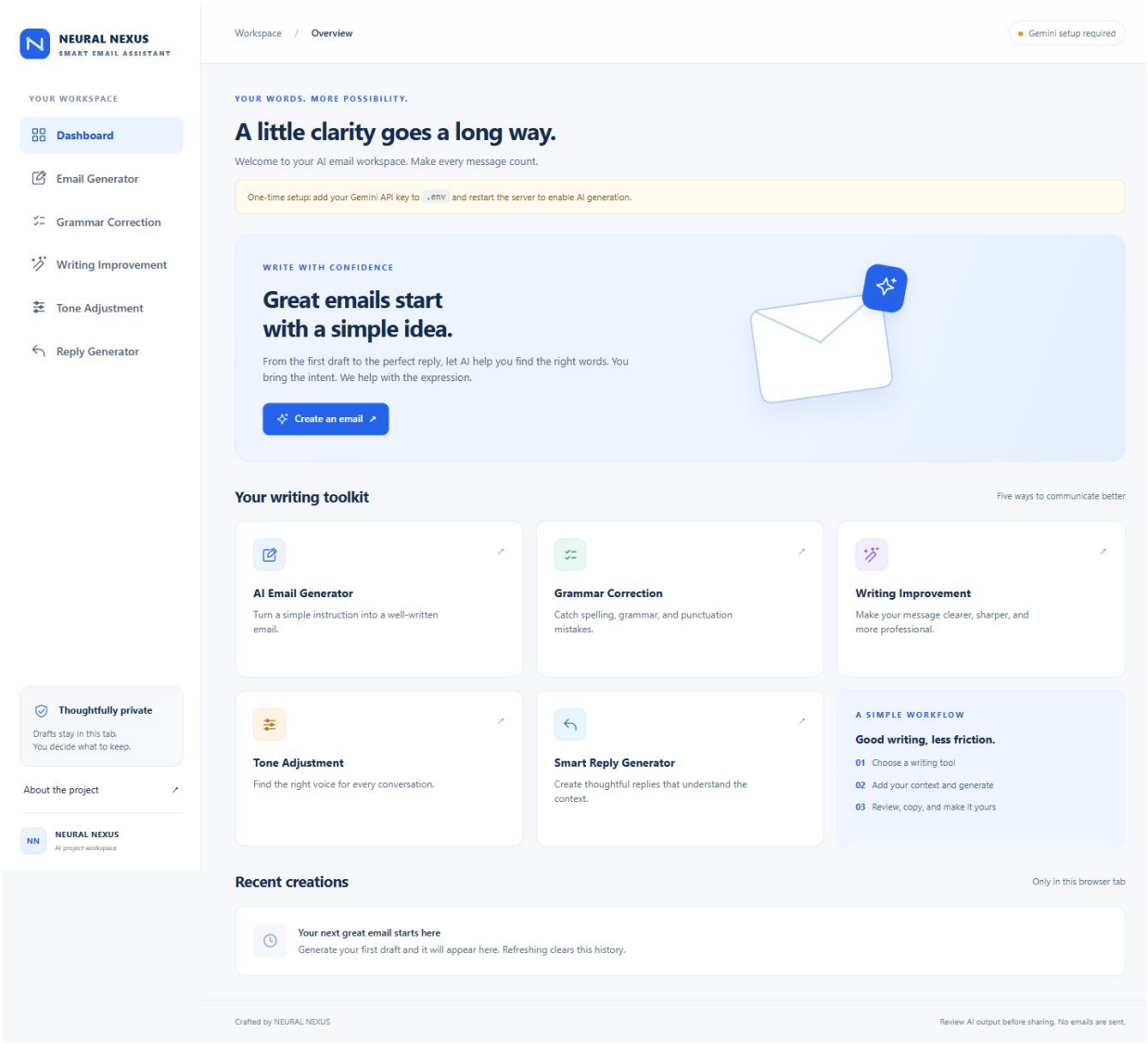


Figure 1. Actual desktop dashboard showing five AI tools, quick access and empty recent history. Configuration notice reflects the absence of an API key.

The dashboard uses a consistent blue-and-white theme, fixed navigation, clear feature cards and a short usage guide. Recent results are intentionally empty until a real generation succeeds.

Responsive design

At narrow widths the sidebar becomes an icon rail and editor panels stack vertically. The About page remains accessible through mobile navigation. Focus styles, labels and a skip link support keyboard use.

14. UI screenshots, continued

Email generator

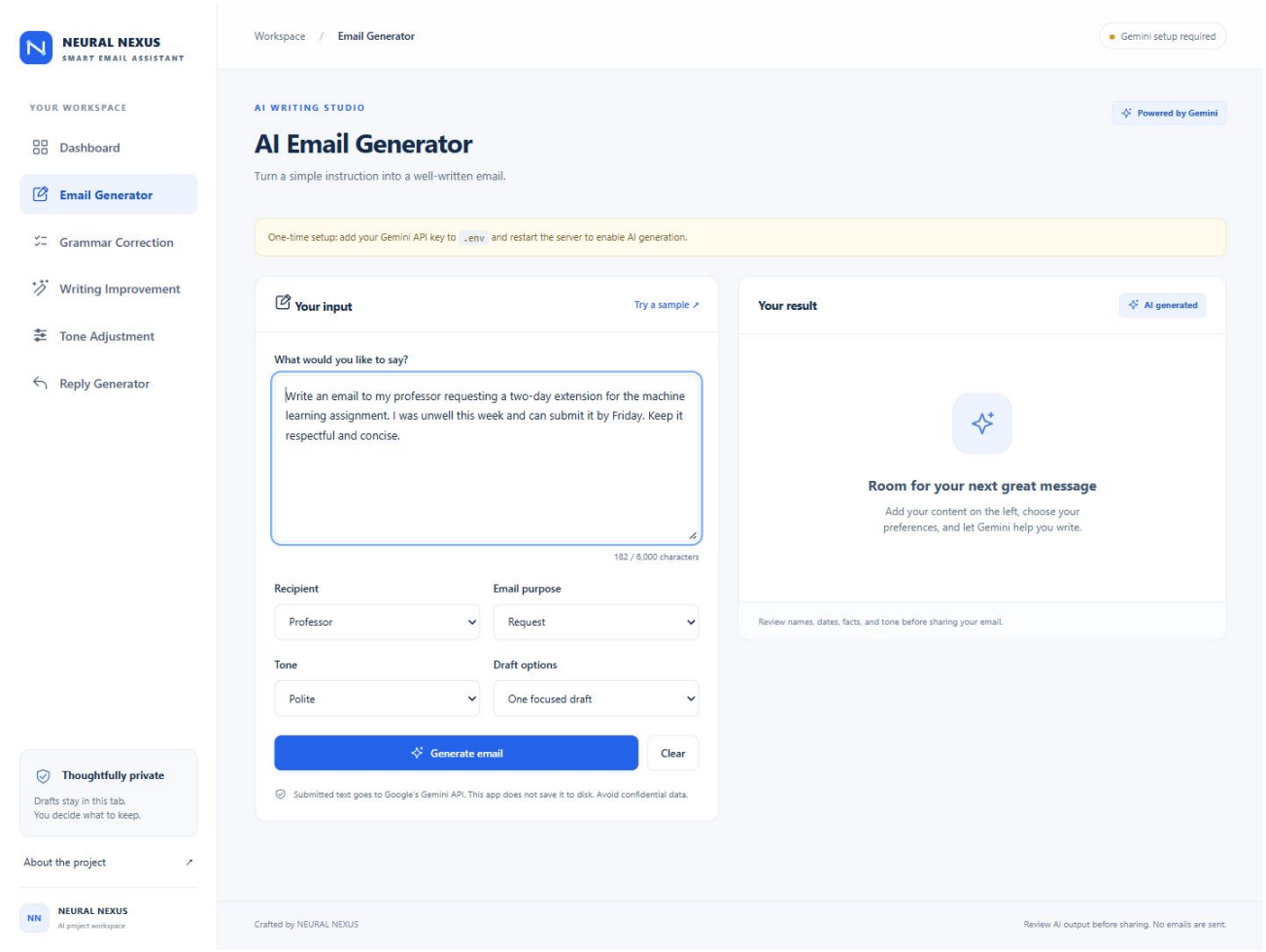


Figure 2. Actual generator with its sample instruction. Recipient, purpose, tone and draft count are available. The result panel is honestly empty before a successful Gemini response.

Grammar, improvement and tone

The other editor pages reuse the same form and results components. Grammar hides tone controls because it preserves the original tone. Improvement uses a professional rewriting objective. Tone adjustment exposes all five choices. Full-resolution captures of every page are included in the screenshots folder.

No fabricated evidence

No API key was provided during delivery. These screenshots document the implemented interface and inputs. They must not be presented as evidence of successful live AI processing. The included live demo guide explains how to capture genuine output after setup.

14. UI screenshots, continued

Context-aware reply

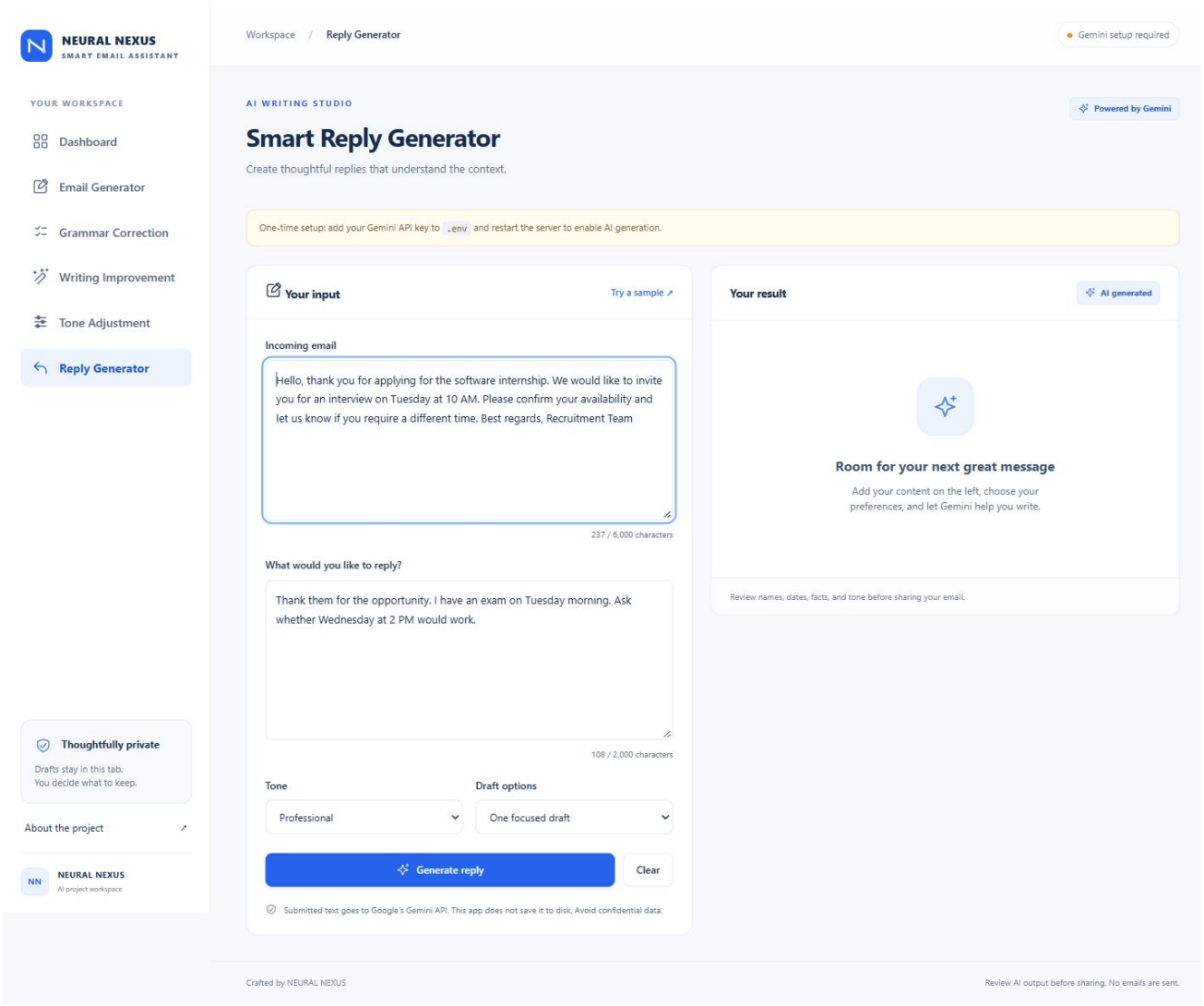


Figure 3. Incoming email and user intent are separate fields. This sample asks to reschedule an interview rather than accept the offered time.

Privacy and result review

The interface states that submitted text goes to Google, that the application does not save it to disk, and that outputs need review. It never claims to access Gmail or send a message. The About page explains the model setting and the difference between key configuration and verified live access.

15. Implementation details

Centralized AI service

AIService exposes generate_email, correct_grammar, improve_writing, adjust_tone and generate_reply. These call a common private execution function with a different operation. The service creates the official GenAI client with a server-side key, timeout and one total attempt, then calls models.generate_content with a dedicated prompt and a Pydantic response schema.

Structured output and validation

The response contract contains intent, drafts, improvements and suggestions. Each draft requires a non-empty subject and body. The service validates the JSON and exact requested draft count. It attaches provider, configured model, local request identifier, elapsed time, token metadata and provider response identifier when available. Metadata supports traceability but is not cryptographic proof.

Prompt engineering

Shared instructions request plain text, preservation of names and dates, no invented commitments and clear placeholders for missing essential details. Separate task prompts specify minimal correction, professional improvement, tone transformation or contextual reply generation. Input is JSON-encoded as data. Prompt boundaries are a safeguard, not a guarantee that the model will always comply.

Frontend components

The frontend uses a shared navigation shell, reusable editor builder, output renderer, icon module and CSS design tokens. Hash navigation preserves in-memory state. Output text is escaped before insertion. Clipboard and Blob download APIs handle local productivity actions without extra model requests. Busy state prevents duplicate clicks and recovers correctly after navigation or failure.

Local security boundary

The launcher binds to loopback and disables access logs. The server restricts hostnames, checks origin and a request token, enforces a 40 KB body limit and adds a restrictive Content Security Policy. This local design must gain authentication, HTTPS and stronger abuse controls before any public hosting.

15. Implementation verification

Check	Observed result
Python automated suite	28 tests passed. SDK calls are mocked only in tests.
Frontend DOM suite	Rendering, escaping, draft switching, copy, download, regenerate, history, clearing, errors and navigation recovery passed.
Real browser checks	All seven pages, sample inputs, missing-key error, clear and mobile navigation inspected.
Live Gemini requests	Not run. No user key supplied. Five-operation demo script included.

Tests validate software behavior, not the quality or availability of a remote model. The current Starlette test client emits an httpx deprecation warning but the checks pass. Browser screenshots and the detailed testing record are included in the package.

16. Advantages

• One workspace covers common drafting and refinement tasks with a consistent interaction model.

• Dedicated prompts reduce the need for users to repeatedly explain formatting requirements.

• Alternative drafts and brief explanations support comparison and informed review.

• A local Python backend protects the API key and keeps infrastructure small.

• Memory-only content reduces unnecessary application storage.

• The source structure makes AI involvement easy to explain during evaluation.

These are design advantages. The project does not claim measured time savings, a quantified accuracy score or superiority over commercial assistants.

17. Limitations

• Live generation requires internet access, a valid key, supported model access and sufficient quota.

• AI can introduce assumptions, change nuance or miss errors. Human review remains necessary.

• Prompt safeguards do not guarantee perfect intent preservation or prompt-injection resistance.

• The app has no permanent draft storage. A refresh clears in-memory work.

• The release has no user accounts, Gmail integration, inbox access or email sending.

• The server is designed for local demonstration, with a small concurrency bound rather than a public service capacity plan.

• No live quality or latency benchmark was conducted without a key.

18. Future enhancements

Future work could add opt-in encrypted draft storage, a reviewed style guide, sentence-level change highlighting and explicit multilingual controls. Evaluation could use a consented dataset with human ratings for intent preservation and factual consistency. A public deployment would need authentication, per-user quotas, secure hosting and operational monitoring.

These items are future proposals, not delivered features. External email account integration remains outside the agreed release scope.

19. Conclusion

NEURAL NEXUS implements a complete local email assistance workflow with generative AI at its center. The application includes five real SDK-backed operations, a custom responsive UI, privacy-conscious local state, safe failure handling and reproducible offline checks. The handover includes code, documentation, presentation assets and a genuine five-operation demo procedure. Adding a valid Gemini key is the remaining user configuration step before live evaluation.

20. References

1. **Google GenAI Python SDK**
https://googleapis.github.io/python-genai/

2. **Gemini 3.5 Flash-Lite model documentation**
https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite

3. **Gemini structured output documentation**
https://ai.google.dev/gemini-api/docs/structured-output

4. **Google AI Studio API keys**
https://aistudio.google.com/apikey

5. **Gemini API terms**
https://ai.google.dev/gemini-api/terms

6. **FastAPI documentation**
https://fastapi.tiangolo.com/

7. **Pydantic documentation**
https://docs.pydantic.dev/latest/

Official SDK and model references checked on November 1, 2026. Product capabilities, access rules and package versions can change. The delivered source and screenshots are the primary evidence for implementation-specific claims.

Windows handover checklist

- Extract the ZIP and open neural-nexus in VS Code.
- Create a Python 3.12 virtual environment and install requirements.txt.
- Copy .env.example to .env and add GEMINI_API_KEY.
- Run .\.venv\Scripts\python.exe run.py and open http://127.0.0.1:8000.
- Follow DEMO_GUIDE.md and run scripts/live_demo.py to verify real outputs.

The README contains exact PowerShell commands, configuration details and troubleshooting. The presentation folder contains the editable 15-slide deck. The screenshots folder contains the actual UI captures used here.